

ROTEIRO 13 – Integração, CI/CD, Containerização e Publicação da Automação no GHCR

Disciplina: Técnicas de Hyperautomation

Professor: Moisés Levy

Semana 10

Foco: Integração dos processos, desenvolvimento em equipe, testes, Docker, CI/CD com GitHub Actions e publicação no GHCR.

1. Objetivos da Atividade

Objetivo Geral

- ✓ Integrar os processos desenvolvidos anteriormente em uma única solução de Hyperautomation, aplicando **Git, GitHub, testes, Docker e CI/CD** para automatizar sua validação, construção e publicação.

Objetivos Específicos

- ✓ Integrar os Processos 3, 4 e 5 aos Processos 1 e 2 existentes;
- ✓ Implementar tratamento de erros, logs, monitoramento e mecanismos de recuperação;
- ✓ Utilizar Git e GitHub para versionamento;
- ✓ Criar testes automatizados;
- ✓ Containerizar a solução com Docker;
- ✓ Configurar CI/CD com GitHub Actions;
- ✓ Publicar e executar a imagem no GHCR.

2. Desenvolvimento da Atividade

A atividade será realizada em equipe. Os **Processos 1 e 2**, desenvolvidos anteriormente, deverão ser mantidos e integrados aos **Processos 3, 4 e 5**.

Ao final, todos os processos deverão funcionar como **uma única solução**, executada pelo arquivo principal:

pai.bot.py

Estrutura geral

projeto-hyperautomation/

```

|
|— processo1/
|— processo2/
|— processo3/
|— processo4/
|— processo5/
|
|— pai.bot.py
|— tests/
|— requirements.txt
|— Dockerfile
|— .gitignore
|
|— .github/
    |— workflows/
        |— ci-cd.yml
    
```

Os Processos 1 e 2 **não deverão ser desenvolvidos novamente**. A equipe deverá reutilizá-los, adaptá-los quando necessário e integrá-los aos novos processos.

3. Processos a Desenvolver e Praticar

Os Processos 3, 4 e 5 deverão ser implementados considerando as regras de negócio já definidas.

Processo	O que praticar
Processo 3 — Cadastro	Integração com API/ML, logs, tratamento de erros e fallback
Processo 4 — SAC	Logs, tratamento de erros, fallback e monitoramento
Processo 5 — Relatórios	Logs, monitoramento, métricas e versionamento
Todos os processos	Integração, testes e execução containerizada com Docker

Processo 3 — Cadastro

Responsável pela execução do cadastro dos clientes.

Fluxo:

Processo 2



Consulta / Validação



Cadastro



Registro do resultado



Processo 4

Deverá contemplar, conforme a necessidade do processo:

- ✓ integração com serviços/API/ML;
- ✓ registro de logs;
- ✓ tratamento de erros;
- ✓ fallback para situações de falha;
- ✓ registro do resultado do cadastro.

Processo 4 — SAC

Responsável pelo tratamento dos resultados e comunicação com o cliente.

Fluxo:

Processo 3



Verificação do status



Tratamento do resultado



Comunicação com o cliente



Processo 5

Deverá contemplar:

- ✓ logs da execução;
- ✓ tratamento de erros;
- ✓ fallback;
- ✓ monitoramento da execução;
- ✓ registro do atendimento.

Processo 5 — Relatórios e Gerência

Responsável pela consolidação das informações e geração dos indicadores.

Fluxo:

Processo 4



Consolidação



Métricas e indicadores



Relatório Gerencial



Processo concluído

Deverá contemplar:

- consolidação dos resultados;
- geração de métricas;
- monitoramento;
- registro de logs;
- versionamento dos resultados/relatórios.

4. Integração Final

Os cinco processos deverão funcionar como uma única solução:

PROCESSO 1



PROCESSO 2



PROCESSO 3 — Cadastro



PROCESSO 4 — SAC



PROCESSO 5 — Relatórios



PROCESSO CONCLUÍDO

A integração deverá ser realizada pelo:

`pai.bot.py`

O arquivo principal será responsável por **orquestrar a execução dos cinco processos na sequência definida pelo negócio.**

5. Git e GitHub

A equipe deverá utilizar **Git e GitHub** durante todo o desenvolvimento.

Deverão ser aplicados:

- ✓ versionamento do código;
- ✓ commits organizados;
- ✓ integração dos processos;
- ✓ registro das alterações.

Exemplos:

feat: implementa processo 3

feat: implementa processo 4

feat: implementa processo 5

feat: integra processos no pai.bot.py

test: adiciona testes

docker: adiciona dockerfile

ci: configura pipeline

cd: configura publicação no ghcr

6. Testes Automatizados

A equipe deverá criar testes para validar os processos e sua integração.

Os testes deverão ser executados automaticamente pelo **GitHub Actions**.

Git Push



GitHub Actions



Instala dependências



Executa testes



Testes aprovados?



Build Docker



Publicação no GHCR

Caso os testes apresentem erro, o pipeline deverá **interromper o processo**, impedindo a publicação da imagem.

7. Containerização com Docker

A solução completa deverá ser containerizada.

A equipe deverá criar um **Dockerfile** na raiz do projeto.

O container deverá iniciar a solução pelo:

pai.bot.py

A execução deverá contemplar os **cinco processos integrados**.

Docker



pai.bot.py



Processos 1 → 2 → 3 → 4 → 5

8. GitHub Actions – CI/CD

A equipe deverá criar:

.github/

└─ workflows/

└─ ci-cd.yml

O workflow deverá automatizar:

CI

- ✓ obtenção do código;
- ✓ configuração do ambiente;
- ✓ instalação das dependências;
- ✓ execução dos testes.

CD

- ✓ construção da imagem Docker;
- ✓ login no GHCR;
- ✓ publicação da imagem.

Código



Testes



Build Docker



GHCR

8.1. Criando o Workflow de CI/CD

Nesta etapa, a equipe irá criar o arquivo responsável por automatizar os testes, a construção da imagem Docker e a publicação no GitHub Container Registry (GHCR).

- ✓ **Importante:** o workflow abaixo já está pronto. A equipe deverá apenas criar o arquivo no local indicado, copiar o código e ajustar o nome do arquivo principal caso necessário.

8.1.1. Criar a pasta do Workflow

Na raiz do projeto, a estrutura deverá ficar assim:

```
projeto-hyperautomation/
├── processo1/
├── processo2/
├── processo3/
├── processo4/
├── processo5/
├── pai.bot.py
├── tests/
├── requirements.txt
├── Dockerfile
├── .gitignore
├── .github/
│   └── workflows/
│       └── ci-cd.yml
```

No VS Code:

1. Crie a pasta **.github**.
2. Dentro dela, crie a pasta **workflows**.
3. Dentro de **workflows**, crie o arquivo: `ci-cd.yml`

8.1.2. Código do CI/CD

Escreva o código abaixo para dentro do arquivo `ci-cd.yml`:

```
name: CI/CD - HyperAutomation

# Executa o workflow quando houver push ou Pull Request

# nas branches main ou develop.

on:

  push:

    branches:

      - main
```

```

- develop

pull_request:

  branches:

    - main

    - develop

jobs:

  # =====

  # CI - INTEGRAÇÃO CONTÍNUA

  # =====

  testes:

    name: Testes e validações

    # Sistema operacional utilizado pelo GitHub Actions

    runs-on: ubuntu-latest

    steps:

      # -----

      # 1. Baixa o código do repositório

      # -----

      - name: Baixar código do repositório

        uses: actions/checkout@v4

      # -----

      # 2. Configura o Python

      # -----

      - name: Configurar Python

```

```

uses: actions/setup-python@v5

with:

  python-version: "3.14"

# -----

# 3. Instala as dependências do projeto

# -----

- name: Instalar dependências

  run: |

    python -m pip install --upgrade pip

    pip install -r requirements.txt

# -----

# 4. Verifica erros de sintaxe nos arquivos Python

# -----

- name: Verificar sintaxe

  run: |

    python -m compileall .

# -----

# 5. Executa os testes automatizados

# -----

- name: Executar testes

  run: |

    python -m pytest -v

# =====

```

```
# CD - ENTREGA CONTÍNUA

# =====

build-and-push:

  name: Build e publicação no GHCR

  # O job só será executado depois que os testes forem aprovados.

  needs: testes

  # Publica a imagem somente quando houver push na branch main.

  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  runs-on: ubuntu-latest

  # Permissões necessárias para publicar a imagem no GHCR.

  permissions:

    contents: read

    packages: write

  steps:

    # -----

    # 1. Baixa o código do repositório

    # -----

    - name: Baixar código do repositório

      uses: actions/checkout@v4

    # -----

    # 2. Faz login no GitHub Container Registry

    # -----

    - name: Login no GHCR
```

```

uses: docker/login-action@v3

with:

  registry: ghcr.io

  username: ${ github.actor }

  password: ${ secrets.GITHUB_TOKEN }

# -----

# 3. Define o nome da imagem Docker

# -----

- name: Definir nome da imagem

run: |

  IMAGE_NAME=ghcr.io/${ github.repository }

  echo "IMAGE_NAME=${IMAGE_NAME,,}" >> $GITHUB_ENV

# -----

# 4. Constrói a imagem Docker

# -----

- name: Construir imagem Docker

run: |

  docker build -t $IMAGE_NAME:latest .

# -----

# 5. Publica a imagem no GHCR

# -----

- name: Publicar imagem no GHCR

run: |

```

```
docker push $IMAGE_NAME:latest
```

8.1.3. O que esse código faz?

O workflow possui duas etapas principais:

CI

- Baixa o código
- Configura Python
- Instala dependências
- Verifica sintaxe
- Executa testes

Se os testes forem aprovados:

CD

- Login no GHCR
- Docker Build
- Docker Push

O fluxo completo é:

```
git push
↓
GitHub Actions
↓
Instala dependências
↓
Testes
↓
Testes aprovados?
↓
Docker Build
↓
GHCR
```

Importante: se os testes falharem, o job build-and-push não será executado e a imagem não será publicada.

8.1.4. Entendendo as partes principais

On: Define quando o workflow será executado.

on:

push:

branches:

- main
- develop

Significa que o **workflow** será iniciado quando houver um push para main ou develop.

Também será executado em Pull Requests destinados a essas branches.

needs: testes

needs: testes

Significa: O processo de build só poderá começar depois que o job de testes terminar com sucesso.

Assim:

```
TESTES
↓
APROVADOS
↓
BUILD
↓
GHCR
```

permissions

```
permissions:
  contents: read
  packages: write
```

Permite que o GitHub Actions leia o código do repositório e tenha permissão para publicar a imagem no GHCR.

GITHUB_TOKEN

```
password: ${ secrets.GITHUB_TOKEN }
```

O GitHub fornece automaticamente esse token durante a execução do workflow.

Não é necessário criar ou colocar manualmente uma senha do GHCR no código.

docker build

`docker build -t $IMAGE_NAME:latest .` Constrói a imagem Docker utilizando o Dockerfile existente na raiz do projeto.

docker push

`docker push $IMAGE_NAME:latest`

Envia a imagem construída para o GitHub Container Registry.

8.2. Enviar o Workflow para o GitHub

Depois de criar o arquivo `ci-cd.yml`, salve o arquivo: `Ctrl + S`

No terminal do VS Code, execute: `git status`

Verifique se o arquivo aparece como novo ou modificado.

Depois: `git add .github/workflows/ci-cd.yml`

Crie o commit: `git commit -m "ci: configura pipeline de testes e publicação no GHCR"`

Envie para o GitHub: `git push`

8.3. Verificar a execução do GitHub Actions

Depois do `git push`, o GitHub deverá iniciar automaticamente o workflow.

No navegador:

Repositório no GitHub



Actions



CI/CD - HyperAutomation

A equipe deverá localizar a execução mais recente.

Será possível visualizar os jobs:

Testes e validações



Build e publicação no GHCR

8.3.1. Verificar os testes

Clique em: Testes e validações

A equipe poderá acompanhar:

- ✓ Baixar código do repositório
- ✓ Configurar Python
- ✓ Instalar dependências
- ✓ Verificar sintaxe
- ✓ Executar testes

Se todos apresentarem sucesso:

- ✓ Success

os testes foram aprovados.

8.3.2. Verificar o Build e a publicação

Depois que os testes forem aprovados, o GitHub Actions executará:

Build e publicação no GHCR

A equipe deverá verificar:

- ✓ Baixar código do repositório
- ✓ Login no GHCR
- ✓ Definir nome da imagem
- ✓ Construir imagem Docker
- ✓ Publicar imagem no GHCR

Se todos apresentarem:

- ✓ Success

a imagem foi construída e publicada com sucesso.

8.4. Onde encontrar a imagem publicada no GHCR

Depois que o workflow terminar com sucesso:

1. Acesse o repositório no GitHub.
2. Na página principal do repositório, localize a área de Packages.
3. A imagem Docker publicada deverá aparecer como um pacote do repositório.

A imagem terá um endereço semelhante a:

ghcr.io/usuario/repositorio:latest

Por exemplo:

ghcr.io/moiseslevy/projeto-hyperautomation:latest

- ✓ **Atenção:** cada equipe deverá utilizar o nome real do seu próprio usuário e repositório. Não copie o exemplo acima.

8.5. Baixar a imagem do GHCR

Para validar a imagem publicada, abra o terminal e execute:

docker pull ghcr.io/USUARIO/REPOSITORIO:latest

Substitua: USUARIO

pelo usuário do GitHub e: REPOSITORIO

pelo nome do repositório.

Exemplo:

docker pull ghcr.io/moiseslevy/projeto-hyperautomation:latest

O Docker fará o download da imagem publicada no GHCR.

8.6. Executar a imagem publicada

Depois do docker pull, execute:

```
docker run --rm ghcr.io/USUARIO/REPOSITORIO:latest
```

Exemplo:

```
docker run --rm ghcr.io/moiseslevy/projeto-hyperautomation:latest
```

O Docker deverá:

```
GHCR
↓
Imagem Docker
↓
Container
↓
pai.bot.py
↓
Processo 1
↓
Processo 2
↓
Processo 3
↓
Processo 4
↓
Processo 5
```

8.7. Se o pacote GHCR estiver privado

Caso o pacote não possa ser baixado publicamente, será necessário realizar login no GHCR antes do docker pull.

```
docker login ghcr.io
```

O terminal solicitará as credenciais necessárias.

- ✓ **Importante:** nunca coloque tokens, senhas ou chaves diretamente no código, no Dockerfile ou no repositório.

Para a atividade, a equipe deverá utilizar as configurações de acesso definidas para o projeto e manter informações sensíveis protegidas.

8.8. O que fazer quando o GitHub Actions apresentar erro?

Se aparecer o status **X Failed**, a equipe **não deve** tentar publicar a imagem manualmente. O primeiro passo obrigatório é identificar a origem da falha seguindo este fluxo no GitHub:

1. Acesse a aba **Actions** no seu repositório do GitHub.
2. Clique no **Workflow** que apresentou o erro.
3. Selecione o **Job** específico que falhou.
4. Na lista de etapas (steps), localize a **Etapa com erro** (marcada com o ícone de falha em vermelho).
5. Clique sobre a etapa para expandir os logs e **leia a mensagem detalhada** exibida no terminal para identificar o problema exato (como dependências ausentes, falhas de sintaxe ou caminhos de arquivos incorretos).

Exemplos Práticos de Resolução

Se a falha ocorrer em uma etapa específica, siga a recomendação correspondente:

- **Se falhar em Instalar dependências:**

- ✓ Verifique o arquivo requirements.txt para garantir que pacotes ou versões estejam corretos.

- **Se falhar em Executar testes:**

- ✓ Execute os testes localmente via terminal para debugar:

```
python -m pytest -v
```

- ✓ Corrija os testes que apresentarem falhas no código.

- **Se falhar em Construir imagem Docker:**

- ✓ Verifique os seguintes arquivos e a estrutura do projeto:

- Dockerfile
- requirements.txt
- Caminhos e estrutura de pastas referenciados.

Após as correções:

Para disparar a pipeline novamente, envie as alterações para o repositório:

```
git add .
git commit -m "fix: corrige pipeline"
git push
```

O GitHub Actions detectará o envio (push) e executará os testes e o build automaticamente.

8.9. Validação final do CI/CD

A equipe deverá comprovar o seguinte fluxo:

```
ALTERAÇÃO NO CÓDIGO
↓
git add
↓
git commit
↓
git push
↓
GITHUB ACTIONS
↓
TESTES
↓
TESTES APROVADOS
↓
DOCKER BUILD
↓
GHCR
↓
docker pull
↓
docker run
↓
pai.bot.py
↓
PROCESSOS 1 → 2 → 3 → 4 → 5
↓
PROCESSO CONCLUÍDO
```

8.10. Checklist da equipe

Antes da demonstração, confirme:

- ✓ O projeto está no GitHub.
- ✓ O arquivo ci-cd.yml está em .github/workflows/.
- ✓ O workflow executa automaticamente após o git push.
- ✓ Os testes são executados pelo GitHub Actions.
- ✓ Os testes foram aprovados.
- ✓ O Docker Build foi executado pelo workflow.
- ✓ A imagem foi publicada no GHCR.
- ✓ A equipe localizou o pacote no GitHub.
- ✓ A imagem foi baixada com docker pull.
- ✓ A imagem foi executada com docker run.
- ✓ O pai.bot.py iniciou a solução.
- ✓ Os cinco processos foram executados.
- ✓ A equipe consegue demonstrar o fluxo completo.

8.11. Fluxo que deverá ser demonstrado pela equipe

Na apresentação final, a equipe deverá demonstrar na prática:

1. Alterar o código
↓
2. git add
↓
3. git commit
↓
4. git push
↓
5. Acessar Actions no GitHub
↓
6. Mostrar os testes
↓
7. Mostrar o Build Docker
↓
8. Mostrar a publicação no GHCR
↓
9. Executar docker pull
↓
10. Executar docker run
↓
11. Demonstrar Processo 1
↓
12. Demonstrar Processo 2
↓
13. Demonstrar Processo 3



14. Demonstrar Processo 4



15. Demonstrar Processo 5



16. Apresentar o resultado final

Resultado esperado: a equipe deverá conseguir explicar e demonstrar que uma alteração no código pode passar automaticamente por testes → build → publicação no GHCR, e que a mesma imagem publicada pode ser posteriormente executada com Docker.

9. Publicação e Execução no GHCR

Após os testes serem aprovados, a imagem deverá ser publicada automaticamente no **GitHub Container Registry (GHCR)**.

Testes aprovados



Build



GHCR



Docker Pull



Docker Run



pai.bot.py



Processos 1 → 2 → 3 → 4 → 5

A equipe deverá comprovar que a solução publicada funciona corretamente dentro do container.

10. Demonstração Final

Cada equipe deverá demonstrar, na prática:

1. Alteração no código;
2. git commit;
3. git push;
4. Execução do GitHub Actions;
5. Execução dos testes;
6. Build da imagem Docker;
7. Publicação no GHCR;
8. docker pull;
9. docker run;
10. Execução integrada dos cinco processos.

Fluxo final

ALTERAÇÃO



GIT PUSH



GITHUB ACTIONS



TESTES



BUILD DOCKER



GHCR



DOCKER PULL



DOCKER RUN



PAI.BOT.PY



PROCESSOS 1 → 2 → 3 → 4 → 5



PROCESSO CONCLUÍDO

11. Ferramentas Utilizadas

- **Python** — desenvolvimento;
- **Git/GitHub** — versionamento e repositório;
- **pytest** — testes;
- **Docker** — containerização;
- **GitHub Actions** — CI/CD;
- **GHCR** — armazenamento da imagem;
- **VS Code** — desenvolvimento.

12. Entregáveis

Ao final, a equipe deverá apresentar:

- ✓ **Cinco processos integrados** e executados pelo `pai.bot.py`;
- ✓ **Repositório GitHub**, testes e Dockerfile organizados;
- ✓ **Workflow CI/CD** executado com sucesso;
- ✓ **Imagem publicada no GHCR** e executada em container;
- ✓ **Evidências e demonstração da solução completa.**

13. Competências Desenvolvidas

Ao concluir a atividade, o aluno deverá ser capaz de:

- ✓ Integrar múltiplos processos de automação;
- ✓ Aplicar logs, tratamento de erros, fallback e monitoramento;
- ✓ Criar testes automatizados;
- ✓ Utilizar Git/GitHub;
- ✓ Containerizar aplicações com Docker;
- ✓ Criar pipelines CI/CD;
- ✓ Publicar e executar imagens no GHCR.

14. Critérios de Avaliação

A atividade será avaliada considerando:

- ✓ Integração e funcionamento dos cinco processos;
- ✓ Qualidade do código, testes automatizados e versionamento;
- ✓ Logs, tratamento de erros, fallback e monitoramento;
- ✓ Containerização e execução com Docker;
- ✓ CI/CD, build e publicação da imagem no GHCR;
- ✓ Demonstração e participação da equipe.

15. Resultado Esperado

Ao final da atividade, a equipe deverá entregar uma **solução de Hyperautomation integrada, testada, containerizada e publicada**, demonstrando o seguinte ciclo:

Processos → Testes → Logs/Erros → Monitoramento → CI → Docker Build → GHCR → Execução

16. Desenvolvimento da Solução

A equipe deverá **reutilizar os Processos 1 e 2**, desenvolvidos anteriormente, e implementar os **Processos 3, 4 e 5**, integrando os cinco processos em uma única solução.

O arquivo bot.py deverá ser o **ponto de entrada da solução**, responsável por integrar e executar o fluxo completo.

Fluxo da solução

```

Processo 1
  ↓
Processo 2
  ↓
Processo 3 - Cadastro
  ↓
Processo 4 - SAC
  ↓
Processo 5 - Relatórios
  ↓
Processo concluído
  
```

A forma de organização dos arquivos poderá ser adaptada pela equipe, desde que os cinco processos estejam integrados e a solução possua um ponto de entrada para sua execução.

16.1 Processo 3 — Cadastro

A equipe deverá implementar o **Processo 3 – Cadastro**, utilizando os dados produzidos pelo Processo 2.

O processo deverá contemplar:

- ✓ Recebimento e validação dos dados;
- ✓ Consulta e verificação de cadastro;
- ✓ Identificação de duplicidades;
- ✓ Realização do cadastro quando necessário;
- ✓ Tratamento de erros;
- ✓ Retorno do resultado ao Processo 4.

O que praticar: integração com API/ML, logs, tratamento de erros e fallback.

16.2 Processo 4 — SAC

A equipe deverá implementar o **Processo 4 – SAC**, utilizando o resultado do Processo 3.

O processo deverá contemplar:

- ✓ Verificação do resultado do cadastro;
- ✓ Tratamento de sucesso e erro;
- ✓ Comunicação com o cliente;
- ✓ Registro do atendimento;
- ✓ Retorno do resultado ao Processo 5.

O que praticar: logs, tratamento de erros, fallback e monitoramento.

16.3 Processo 5 — Relatórios e Gerência

A equipe deverá implementar o **Processo 5 – Relatórios e Gerência**, consolidando os resultados dos processos anteriores.

O processo deverá contemplar:

- ✓ Consolidação dos resultados;
- ✓ Geração de indicadores;
- ✓ Identificação de sucessos e erros;
- ✓ Registro das principais ocorrências;
- ✓ Geração e apresentação do relatório final.

O que praticar: logs, monitoramento, métricas e versionamento.

17. Integração dos Cinco Processos

Os cinco processos deverão funcionar como **uma única solução**, respeitando a sequência:

```

bot.py
↓
Processo 1
↓
Processo 2
↓
Processo 3
↓
Processo 4
↓
Processo 5
↓
Resultado final + Logs
    
```

O bot.py deverá passar os resultados de cada processo para o processo seguinte.

Importante: os Processos 1 e 2 deverão ser **reutilizados e adaptados quando necessário**. Os Processos 3, 4 e 5 deverão ser desenvolvidos e integrados à solução existente.

18. Divisão das Tarefas da Equipe

Cada integrante deverá desenvolver e testar uma parte da solução.

Responsável	Atividade
Integrante 1	Processo 3 + testes
Integrante 2	Processo 4 + testes
Integrante 3	Processo 5 + testes
Integrante 4	Integração no bot.py + teste de integração
Equipe	CI/CD + Docker + GHCR + validação final

Importante: todos os integrantes deverão participar da integração, testes e validação final. O responsável pela integração não deverá desenvolver sozinho toda a solução.

19. Testes Automatizados

A equipe deverá criar testes automatizados para os processos desenvolvidos e para a integração da solução.

Os testes deverão verificar:

- ✓ Entradas válidas e inválidas;
- ✓ Regras de negócio;
- ✓ Resultados de sucesso;
- ✓ Tratamento de erros;
- ✓ Duplicidades, quando aplicável;
- ✓ Integração entre os processos.

A estrutura poderá seguir, por exemplo:

```
tests/
├── test_modulo1.py
├── test_modulo2.py
├── test_modulo3.py
├── test_modulo4.py
├── test_modulo5.py
└── test_bot.py
```

Executar todos os testes:

```
python -m pytest -v
```

A equipe deverá verificar a cobertura dos testes:

```
python -m pytest --cov=. --cov-report=term-missing
```

A cobertura deverá ser utilizada para identificar partes importantes do código que ainda não possuem testes.

Não é obrigatório atingir 100% de cobertura.

20. Logs, Tratamento de Erros, Fallback e Monitoramento

A solução deverá possuir mecanismos que aumentem a **confiabilidade da automação**.

A equipe deverá demonstrar:

- ✓ Registro das execuções;
- ✓ Registro de sucessos e erros;
- ✓ Tratamento de exceções;
- ✓ **Fallback**, quando aplicável;
- ✓ **Monitoramento** das principais informações da execução.

Os registros deverão permitir identificar, no mínimo:

```
Data/Hora → Processo → Status → Erro/Motivo
```

21. Execução e Validação da Integração

Após desenvolver e testar os módulos, a equipe deverá executar a solução completa:

```
python bot.py
```

A equipe deverá verificar se:

```
Processo 1
  ↓
Processo 2
  ↓
Processo 3
  ↓
Processo 4
```

↓
Processo 5

funcionam corretamente em sequência.

Caso ocorra um erro, a equipe deverá identificar o processo responsável, corrigir o problema e executar novamente os testes.

22. CI/CD com GitHub Actions

Nesta etapa, a equipe irá configurar um **pipeline de CI/CD** para automatizar a validação, construção e publicação da aplicação.

Importante: o workflow abaixo será utilizado como modelo pela equipe. Os alunos deverão entender sua função, adaptar apenas o que for necessário e verificar sua execução no GitHub Actions.

O fluxo será:

```
Git Push
↓
GitHub Actions
↓
Instalar dependências
↓
Executar testes
↓
Docker Build
↓
Publicar imagem no GHCR
```

O que será feito automaticamente?

CI — Integração Contínua

- Instalar as dependências;
- Verificar a sintaxe;
- Executar os testes.

CD — Entrega/Implantação Contínua

- ✓ Construir a imagem Docker;
- ✓ Publicar a imagem no GHCR.

23. Criar o Workflow

Dentro da raiz do projeto, crie a seguinte estrutura:

```
.github/
├── workflows/
│   └── ci-cd.yml
```

No VS Code, você poderá criar as pastas e o arquivo manualmente.

Ou, pelo terminal do VS Code:

```
mkdir .github
mkdir .github\workflows
ni .github\workflows\ci-cd.yml
```

Abra o arquivo:

```
.github/workflows/ci-cd.yml
```

e coloque o código abaixo.

24. Código do CI/CD

```
name: CI/CD - HyperAutomation
  python-version: "3.14"
# 3. Instala as dependências
- name: Instalar dependências
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt
# 4. Verifica erros de sintaxe
- name: Verificar sintaxe
  run: |
    python -m compileall .

# 5. Executa os testes automatizados
- name: Executar testes
  run: |
    python -m pytest -v
```

```
# =====
# CD - BUILD E PUBLICAÇÃO NO GHCR
# =====

publicar:
  name: Build e Publicação no GHCR
  # O job de publicação só começa se os testes
  # forem concluídos com sucesso
  needs: testes
  runs-on: ubuntu-latest
  # Permissões necessárias para publicar a imagem
  permissions:
    contents: read
    packages: write
  steps:
    # 1. Baixa o código
    - name: Baixar código
      uses: actions/checkout@v4
    # 2. Faz login no GHCR
    #
    # O GitHub fornece automaticamente o token
    # GITHUB_TOKEN para autenticação.
    - name: Login no GHCR
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
    # 3. Constrói a imagem Docker
    - name: Construir imagem Docker
      run: |
        docker build \
          -t ${{ env.IMAGE_NAME }}:latest \
          .
    # 4. Publica a imagem no GHCR
```

```
- name: Publicar imagem no GHCR
run: |
  docker push ${env.IMAGE_NAME}:latest
```

25. Entendendo o Workflow

Não é necessário memorizar o código. É importante entender **o que cada parte faz**.

name

```
name: CI/CD - HyperAutomation
```

Define o nome que aparecerá no GitHub Actions.

on

Define **quando o workflow será executado**.

Neste projeto:

```
push:
  branches:
    - main
    - develop
```

Significa que o workflow será executado quando houver um push para main ou develop.

Também será executado em Pull Requests:

```
pull_request:
  branches:
    - main
    - develop
```

jobs

Define os trabalhos que serão executados.

Neste projeto existem dois:

```
testes
↓
publicar
```

needs: testes

needs: testes

Essa é uma parte importante.

Significa:

O job publicar só poderá executar se o job testes terminar com sucesso.

Portanto:

```
Testes passaram?
|
SIM
↓
Docker Build
↓
GHCRC
```

Se os testes falharem:

```
Testes falharam
↓
Pipeline interrompido
↓
Imagem não publicada
```

26. Salvar e Enviar o Workflow para o GitHub

Depois de criar o arquivo:

```
.github/workflows/ci-cd.yml
```

salve:

Ctrl + S

Depois, no terminal:

```
git status
```

Verifique se o arquivo aparece como novo.

Depois:

```
git add .github/workflows/ci-cd.yml
```

Crie o commit:

```
git commit -m "ci: adiciona pipeline de CI/CD"
```

Envie para o GitHub:

```
git push
```

27. Acessar o GitHub Actions

Depois do git push, entre no **repositório do projeto no GitHub**.

Na página do repositório, clique em:

Actions

Você verá o workflow:

CI/CD - HyperAutomation

Clique nele.

Será possível visualizar a execução do pipeline.

28. Verificar a Execução do Pipeline

Ao abrir a execução, serão apresentados os jobs:

- ✓ Testes e Validação
- ✓ Build e Publicação no GHCR

Quando tudo estiver correto, os jobs aparecerão como concluídos com sucesso.

Clique em:

Testes e Validação

para visualizar cada etapa:

```
Baixar código
↓
Configurar Python
↓
Instalar dependências
↓
Verificar sintaxe
↓
Executar testes
```

Depois clique em:

Build e Publicação no GHCR

para verificar:

```
Baixar código
↓
Login no GHCR
↓
Construir imagem Docker
↓
Publicar imagem no GHCR
```

29. Quando o Pipeline Falhar

Se aparecer:

X Failed

a equipe deverá clicar no job que apresentou erro.

Depois deverá localizar a etapa que aparece em vermelho.

Por exemplo:

```
✓ Baixar código
✓ Configurar Python
✓ Instalar dependências
X Executar testes
```

Nesse caso, o problema está nos testes.

A equipe deverá:

1. Ler a mensagem de erro;
2. Corrigir o código;
3. Executar os testes localmente;
4. Fazer um novo commit;
5. Fazer git push;
6. Verificar novamente o GitHub Actions.

O pipeline será executado novamente automaticamente.

30. Containerização com Docker

Antes de utilizar o CI/CD para publicar a imagem, a equipe deverá verificar se o projeto funciona localmente com Docker.

Crie na raiz do projeto:

Dockerfile

Utilize como referência:

```
FROM python:3.14-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "bot.py"]
```

O Dockerfile deverá iniciar o arquivo principal:

bot.py

e executar a solução completa.

31. Criar o .dockerignore

Crie na raiz:

.dockerignore

Exemplo:

```
.git
.github
.pytest_cache
__pycache__
*.pyc
.env
credenciais
```

Nunca coloque senhas, tokens ou chaves de API dentro da imagem Docker.

32. Testar o Docker Localmente

Antes de publicar a imagem, construa-a:

```
docker build -t hyperautomation:1.0 .
```

Depois execute:

```
docker run --rm hyperautomation:1.0
```

A equipe deverá verificar se o container executa:

```
bot.py
↓
Processo 1
↓
Processo 2
↓
Processo 3
↓
Processo 4
↓
Processo 5
```

Importante: se o Docker não funcionar localmente, corrija o problema antes de tentar publicar no GHCR.

33. Publicação da Imagem no GHCR

O **GHCR — GitHub Container Registry** será utilizado para armazenar a imagem Docker do projeto.

A publicação será realizada automaticamente pelo GitHub Actions.

A equipe **não precisará utilizar Docker Hub** nesta atividade.

O workflow fará:

```
GitHub Actions
↓
Docker Build
↓
Login no GHCR
↓
Docker Push
↓
GHCR
```

34. Onde Encontrar a Imagem no GitHub

Depois que o workflow terminar com sucesso, acesse o repositório no GitHub.

Na página principal do repositório, procure a seção:

Packages

A imagem publicada deverá aparecer como um pacote do projeto.

Outra forma de localizar:

```
Repositório
↓
Packages
↓
Imagem Docker
```

A equipe deverá verificar se a imagem foi publicada corretamente.

35. Identificação da Imagem no GHCR

A imagem terá como referência:

```
ghcr.io/USUARIO/REPOSITARIO:latest
```

Por exemplo:

```
ghcr.io/equipe15/hyperautomation:latest
```

Atenção: não escreva exemplo literalmente.

O nome será formado de acordo com o **usuário/organização e o nome do repositório da equipe**.

No workflow, isso é definido automaticamente por:

```
IMAGE_NAME: ghcr.io/${{ github.repository }}
```

Assim, cada equipe poderá utilizar seu próprio repositório sem precisar alterar manualmente o nome da imagem.

36. Validar a Imagem Publicada

Após o workflow terminar com sucesso, a equipe deverá comprovar que a imagem foi publicada no GHCR.

A execução deverá utilizar a imagem publicada.

Para baixar a imagem:

```
docker pull ghcr.io/USUARIO/REPOSITORIO:latest
```

Depois:

```
docker run --rm ghcr.io/USUARIO/REPOSITORIO:latest
```

Substitua:

USUARIO

e

REPOSITORIO

pelos dados reais da equipe.

O objetivo é verificar se a **mesma imagem publicada no GHCR** consegue executar a solução.

37. Fluxo Completo do Projeto

Ao final, a equipe deverá compreender e executar este fluxo:

```

DESENVOLVIMENTO
↓
TESTES LOCAIS
↓
INTEGRAÇÃO DOS 5 PROCESSOS
↓
LOGS / ERROS / FALLBACK / MONITORAMENTO
↓
DOCKER LOCAL
↓
GIT PUSH
↓
GITHUB ACTIONS
↓
TESTES AUTOMATIZADOS
↓
DOCKER BUILD
↓
GHCR
↓
IMAGEM PUBLICADA
↓
DOCKER RUN
↓
EXECUÇÃO DA SOLUÇÃO
    
```

38. Checklist Final

Antes da apresentação, a equipe deverá verificar:

Desenvolvimento

- ✓ ☐ Processos 1 e 2 reutilizados;
- ✓ ☐ Processo 3 desenvolvido;
- ✓ ☐ Processo 4 desenvolvido;
- ✓ ☐ Processo 5 desenvolvido;
- ✓ ☐ Cinco processos integrados no bot.py.

Testes e confiabilidade

- ✓ ☐ Testes automatizados criados;
- ✓ ☐ Testes executados com sucesso;
- ✓ ☐ Cobertura verificada;
- ✓ ☐ Logs implementados;
- ✓ ☐ Tratamento de erros implementado;
- ✓ ☐ Fallback implementado, quando aplicável;
- ✓ ☐ Monitoramento implementado.

Docker

- ✓ Dockerfile criado;
- ✓ dockerignore criado;
- ✓ requirements.txt atualizado;
- ✓ docker build executado com sucesso;
- ✓ docker run executado com sucesso.

CI/CD

- ✓ ci-cd.yml criado;
- ✓ Workflow enviado ao GitHub;

- ✓ GitHub Actions executado;
- ✓ Testes executados automaticamente
- ✓ Docker Build executado automaticamente;
- ✓ Imagem publicada no GHCR.

Validação

- ✓ ☐ Imagem encontrada no GHCR;
- ✓ ☐ Imagem publicada executada;
- ✓ ☐ Cinco processos executados;
- ✓ ☐ Resultado final apresentado.

39. Resultado Final Esperado

A equipe deverá demonstrar uma solução na qual:

- ✓ Os cinco processos estão integrados;
- ✓ Os processos possuem testes automatizados;
- ✓ A solução possui logs, tratamento de erros, fallback e monitoramento;
- ✓ A aplicação funciona em Docker;
- ✓ O GitHub Actions executa os testes automaticamente;
- ✓ O GitHub Actions constrói a imagem Docker;
- ✓ A imagem é publicada automaticamente no GHCR;
- ✓ A imagem publicada consegue executar a solução completa.

Fluxo final

```

PROCESSOS
  ↓
TESTES
  ↓
LOGS / ERROS / FALLBACK / MONITORAMENTO
  ↓
GITHUB ACTIONS
  ↓
TESTES APROVADOS
  ↓
DOCKER BUILD
  ↓
GHCR
  ↓
DOCKER RUN
  
```

↓
PROCESSOS 1 → 2 → 3 → 4 → 5